公开技术说明 • 2026-10

站在 KVMem 的肩膀上

异步页流水：从“装得下”到“跑得动”的探索与实践

致谢 KVMem 原作者团队：Di Chai · Leye Wang · Zeshen Su · Zhiguo Xia · Zhihang Yu（arXiv 2609.04852）

KV 虚拟化
先行者

异步页流水
我们

持续跑得动
目标

THANKS FIRST

首先，致敬原作者的开创性工作

他们在没有先例可循的情况下回答了一个被认为做不到的问题：

“消费级 16GB 显卡，能不能承载百万 token 级的 Agent 工作区？”

论文与创新

KVMem: Virtualizing
Million-Token Agent Workspaces
on a Consumer GPU
(arXiv 2609.04852 · B 站 FirePKU)

开源工程

kvmem/kvmem：
双后端正式化（llama.cpp + NInfer）
Windows RTX 50 正式运行包
完整的验收与失败记录

生态血统

沈三殊——最早把 NInfer × KVMem 结合
tancau——Ring 环实现 · 直接讨论
帅气的 zxz——5060Ti 线 · 融合计划
llama.cpp · Neroued/ninfer ·
iamwavecut/ninfer-all

我们的线始于沈三殊的 NInfer × KVMem 组合；感谢 tancau 与帅气的 zxz 的直接交流与同行；近期 KVMem 官方亦将 NInfer 纳入正式后端——技术收敛。

THEIR INSIGHT

原作者的核心洞察：每一步，只需看见一小部分

Agent 场景把上下文推到几十万到上百万 token（100 万 token 的 KV ≈ 32 GiB，消费卡只有 16 GB）——但模型每一步真正“看见”的 KV 只是一小部分：

Sink（固定开头）

如 12K token · 常驻

Recent（最近窗口）

如 1K token · 常驻

其余全部历史

按检索命中才换入 GPU

→ 于是：完整 KV 存放 Host RAM（可选下沉 SSD），GPU 只保留“当前用得着的”——KV 虚拟化诞生

THEIR WORK

开创性技术与公开进展

系统性实践的首创技术

- KV 虚拟化（含量化附属 + MTP KV 一起分层）
- 检索驱动的按需换入（不是全量搬运）
- LRFU 历史加权 + GQA max 打分
- 所有权移动（恢复不复制、不重新量化）
- 配额 + 版本化发布 + 读租约
- 跨后端统一内存契约（双引擎共用）

公开进展（0.18.0 / RC3）

双后端正式化

llama.cpp + NInfer；Windows RTX 50 (sm120a) 正式包

RC3 实测（5060 Ti 16GB + IQ3_S 模型）

长历史工具轮 53.24 tok/s · 输入 26 万 token

日常文本 50.08 / 代码 64.54 tok/s

整卡显存峰值 ≈ 14.12 GiB

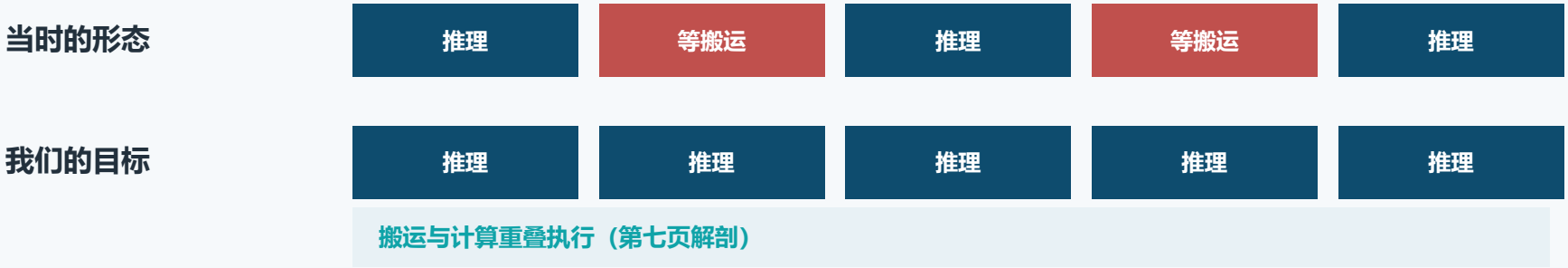
工程纪律示范

配对统计（95% 上界）· 边界“明确拒绝而非猜测”

完整的验收与失败记录公开

THE WALL

留下的那道墙：同步换页



同步换页的推论： “池不要小于工作集”的工程红线，本质是在**回避换页**——池盖住需求就不换。但 Agent 的真实上下文会持续增长，回避终有尽头。

我们的探索从这道墙开始：把“回避换页”变成“驾驭换页”。

OUR EXPLORATION ①

异步页流水（Async Page Pipeline）：让搬运和计算同时发生



页 + 批

64 token/页；8 页一批
控制单次搬运占用

提交即返回

解码步不等搬运
这是“异步”的本质

在飞页计数

“已离开设备、未发布”的
中间态如实记账

publish / drop

搬完校验才发布
确定无用才丢弃

ANATOMY

搬运是怎么“藏”起来的?

主厨

GPU 计算单元（SM）

跑解码的整张计算图
每个解码步约 27-32 ms

专职跑腿

GPU 拷贝引擎（DMA）

显存 ↔ 内存之间搬页
每批 8 页约 1-3 ms

经理

Host 调度线程

三道过滤器：保护范围 → 水位 → 相关性打分
微秒级——不亲手搬

不是让主厨停下活去仓库取菜，而是后厨一直开火、跑腿小哥专职往返仓库与灶台——两双不同的手，同时干活。

1-3 ms 藏在 27-32 ms 里 → 完全隐藏　|　74.4K 深度超池 → 114.4 tok/s

双保险：决策更准（保护/水位/打分三道过滤器）+ 失误更便宜（异步隐藏 + A2 无损）；在飞计数 + 账本保证不丢失、不重复。

OUR EXPLORATION ②

目标函数修正：池从“容量门槛”变成“旋钮”

异步之后，池的收益曲线出现三段结构（200K 上下文场景）：

池 < 工作集（~42K）	甜点区 42K-56K	优选段 56K-96K	> 96K（请求线上）
每步都在搬运——全靠流水硬扛	常规轮次零搬运	覆盖大多数会话	边际收益骤降

实测：池 64K vs 51K 仅多撑约 8 轮会话——过了工作集线后，多买的池主要是“防深堆积”，而深堆积有流水兜底。

16GB 显存是零和的：每 1 GiB 给池，就少 1 GiB 给 KV 精度或权重档位——而精度影响每一轮。→ 浅池 + 深流水 + 高精度。

OUR EXPLORATION ③

正确性工程：给“跑腿”立三条规矩

异步 = 请了跑腿小哥（见第 7 页）。规矩立不好会丢件、会搬错——所以我们给他立三条规矩，外加一张“考卷”：

规矩一：正在用的书不能搬：引擎的“施工图”在启动时就画定（捕获期），运行中不许改——要改就重启，边界清清楚楚

规矩二：货没交稳，旧的不扔：新副本发布之前，旧的那一份永远不许丢——任何时候都有一份“唯一有效”的数据在手

规矩三：订单取消，已出发的必须跑完：请求可以取消，但已经在路上的搬运必须排空才能收账——不收“半路失踪”的账

那张“考卷”（无损门 A2）：同一道题，“池内”与“首次超池”两条路径的答卷**逐字相同**——异步只改变搬运时机，不改变模型输出。

诚实披露：慢峰时搬运会“藏不住”（单批接近步窗）——这就是流水的击穿边界，解法是深流水（见问题页）。

STATUS

已实现的阶段：V2 上线初期的成果快照（2026-10-11~14）

项目	状态
页级异步流水（搬运与解码并行）	已交付并转正（日常运行档）
池标定	81,920 token 实测（16GB 卡低比特 KV 档）
速度	极值 400+ tok/s；1 小时综合平均 220 tok/s
深超池	74.4K 上下文（池约 65K）：114.4 tok/s
无损性	池内 vs 超池输出逐字相同（A2 门）
验证场景	单路长上下文 Agent 会话（多路场景待覆盖）

↑ V2 上线初期的成果（10-11 上线当天：首战峰值 333、超池 103-195 tok/s；随后数日陆续补全上表数字）——**不是最新状态**；最新进展（死亡谷攻关 / 深轮 AB 铁证）见第 13-14 页。

阶段定位：第一阶段（可行性与稳定性）完成——从“能跑”到“长跑不崩、输出无损”；正在进入第二阶段（性能深化）。

MILESTONES

攻关里程碑：异步页流水诞生记（单日闭环）

凌晨 • 问题定义	页漂移：34 轮后 330 页滞留 Host —— decode 掉到 3 tok/s
14:35-17:39 • 三个施工	取消检查点前移 → 批量搬运（8 页/批）→ 实测定案：真凶 = 同步 D2H 拷贝（双峰 10-20×）
20:36 • 异步搬运上线	transfer stream 异步提交 + 下一步收割（约 600 行）——当天首战：峰值 333 tok/s
夜间 • 验证闭环	超池解码 103-195 tok/s（此前 4-6）——提升 20-40×；连续 10+ 轮零降速
10-13 • 转正	独立交叉审核（2×40 轮深压）+ 真实长会话 → 默认基线：127 轮 0×500 · 峰值 403.7 tok/s

单日完成 “问题定义 → 施工 → 定案 → 上线 → 验证”；每一步都带 A/B 回退臂（NINFER_ASYNC_DEMOTE=0）。

OPEN QUEST

一个仍在攻关的问题：时段性慢态（“死亡谷”）

现象与指纹

- 偶发延迟尖峰：同一步耗时可从 30 ms 级 → 数百 ms 级（深端可达秒级）
- 指纹：GPU 满占用 / 满频，但功耗极低、显存几乎不动
- 分钟级自愈；输出逐字无损（与池内路径相同）
- 曾以“小时级窗口”极难复现——现已可按需制造（见右）

攻关进展（最新）

- 按需复现：外部显存压力（约 1.2 GiB）⇒ 施加即入谷（速度降至 1/25）、释放即恢复、可重复
- 机制定位：时间全在 host 侧提交调用（submit≈wall、wait=sync≈0）⇒ 驱动内部阻塞
- 排除清单：PDL ✕ 出局（原最大嫌疑）· 页中转 ✕ · 宿主路径 ✕ · 页档位 ✕ · 池大小 ✕ · 节流 ✕
- 应对：避免与引擎争抢显存（唯一确证有效）；最小复现协议已备（供上游）

机制已收敛：整轮时间全部花在 host 侧提交调用里（submit ≈ wall、wait = sync ≈ 0）⇒ 卡点在引擎自身的提交路径（驱动内部阻塞）——外部算力 / 拷贝 / 节流全部正常。

里程碑：可按需复现（约 1.2 GiB 外部显存压力）；3 步、约 2 分钟的最小复现协议已备好供上游 / 驱动侧研判。仍待解：自然状态下的触发源。

MANHUNT

追捕进行时：真凶未获，但包围圈在收紧

追凶已连续 **120 小时（5 昼夜）不间断**（10-11 → 10-16）：全程工程化受控压测 + 现场取证；中途顺手修复和改进了大量问题（仪表缺陷、回归、自伤 bug 等）。今天（10-16）再次进行异步页流水极深池 AB 测试——结果见下页。

异步的贡献：同条件对照铁证

- 深轮重载（2.3-3.5× 超池）拆掉异步：decode 拖慢约 20%（r=0.80）
- 最深段（3.5× 超池）：≈ 2.0 倍差（235.6 vs 119.4 tok/s）
- 停机（ring STALL）：33 → 2 次（约 16×）
- 浅载/单发：打平（r=1.13）⇒ 口径：“看工作集深度”

嫌疑范围：不断收紧的排除法

- 已出局：PDL（原最大嫌疑）· 页中转 · 宿主路径 · 页档位 · 池大小 · 驱动节流——逐项实测
- 已锁定：时间全在 host 侧提交调用（submit≈wall）⇒ 驱动内部阻塞
- 已可按需复现（外部显存压力协议）：不再“等窗口”
- 现存嫌疑人：驱动提交路径内部 · 自然触发源 · 叠加态假说

真凶未获，但猎捕方式已从“等它出现”变成“随时造它出来”——剩下的问题：把提交路径上的具体阻塞点钉到行号级（驱动侧定位 + 现场取证并行推进）。

备注：异步收益的对照证据口径 = n=3/臂、单机、健康期；谷期影响未覆盖（留作后续）。

EVIDENCE

异步贡献的判决实验：极深池 AB 对照（2026-10-16）

臂	run	round1（1.15× 超池）	round2（2.3×）	round3（3.5×）	深段均值(r2,r3)
OFF（同步回退）	1	327.6	218.9	228.1	223.5
OFF	2	332.0	191.5	107.5	149.5
OFF	3	325.8	256.7	119.4	188.1
ON（异步）	1	293.9	285.7	237.3	261.5
ON	2	303.1	264.2	125.8	195.0
ON	3	330.1	234.2	235.6	234.9

实验数据：上下文 ~75.3K → ~152K → ~229K token（池 65,536 ⇒ 超池 1.15× / 2.3× / 3.5×）；每轮长输出 max 2000 token；轮间前缀缓存部分复用（49.3% / 66.4%）；两臂仅差 NINFER_ASYNC_DEMOTE 开关

主判据：OFF 深段 median 188.1 vs ON 234.9 ⇒ r = 0.80 < 0.85：异步净收益成立；三轮配对 ON 全胜

最深段（3.5× 超池）：119.4 vs 235.6 ≈ 2.0 倍；停机（ring STALL）33 → 2 次（约 16×）

浅载对照（同日单发）：r = 1.13 打平 ⇒ 口径：价值随工作集深度上升而显现——异步正是为深载设计的

诚实备注：n=3/臂、单机、健康期；同步臂未“死”——历史“v2 前多轮必死”的更强主张未复现（观测为“可用但慢”）；**且本次未现死亡谷，可能因前序多轮修复已碰巧缓解了该状态**（未在对照中复现，待后续验证）。

OPEN ISSUES

其余问题（诚实清单）

流水击穿边界

慢峰单批搬运 11.5-33.7 ms 会接近/超过解码步窗——异步方案固有的暴露窗口

深流水尚未转正

多批同时搬运在飞是摊薄击穿的方向，仍在验证中

SSD 层：有意取舍

全部 KV 可常驻 Host RAM（最坏 ≈8.6 GB / 本机 128 GB）——无容量必要，热成本 +11% 不划算

拷贝式搬运

仍是拷贝；原作者已实现“所有权移动”——追赶中。落地后用按需协议做拷贝版 vs 移动版对照（入谷率/谷深），并测对暴露面的作用

覆盖差异

我们深耕单路长上下文；原作者的多路并发场景我们覆盖不足

注：“时段性慢态”已在上一页单独展开。

ROADMAP

未来的方向

1	**深流水**	多批在飞，把击穿余量拿回来——“浅池+深流水+高精度”最后一环
2	**所有权移动**	向原作者语义看齐：搬运不复制、恢复零快照
3	**Host 层深化**	扩大 Host 配额 + 优化 RAM↔显存搬运；SSD 按需再评估（非跟随项）
4	**慢态收口**	驱动提交阻塞的上游定位（3 步最小复现已备）；自然触发源追查
5	**误搬率观测**	统计“搬出后又快速搬回”的比率——让决策质量从猜变成量
6	**向上游回馈**	修复与实测数据贡献回社区（含与 KVMem 官方后端的收敛对接）

WHY OPEN SOURCE

为什么要开源这份工作

缘起：到火候了就开门

- 技术已达“限定场景生产可用”：单路长上下文 Agent 会话（显存独占）实跑验证通过
- 但最大未解课题（时段性慢态）已超出单线攻关的带宽——
- **开源 = 借社区 / 上游 / 驱动侧协同攻关**
- 我们已备好”换帮助”的钥匙：约 2 分钟的最小复现协议（可随时造出该状态）

定位与承诺（口径先行）

- 定位：“研究性工程 + 限定场景生产可用”；多路并发未覆盖
- 承诺 ① 进谷可控：指纹可检测 / 分钟级自愈 / 回避手段已确证
- 承诺 ② 输出无损：池内 vs 超池逐字相同（A2 门）
- 不承诺：永不进谷（触发源未解）；通用生产级

我们希望换到的：不是“夸赞”，而是**同路人的线索与判决**——谁在驱动侧见过提交阻塞、谁复现过同样的慢态、谁能一起把行号级的根因钉死。

说明：本页口径来自发布前检查清单（docs/发布前事项-...-20261016.md）§0 与缘起段。

SUMMARY & THANKS

三句话

1. 原作者把“装不下”变成“装得下”——KV 虚拟化让消费卡承载百万 token 工作区；

2. 我们在其基础上把“换页”从同步等待变成异步页流水——搬运与计算重叠，超池不等于减速；

3. 第一阶段已交付、慢态攻关在进行——目标：从“持续装得下”到“持续跑得快”。

再次感谢：KVMem 作者团队（B 站 FirePKU）· 沈三殊 · tancau（Ring 环）· 帅气的 zxz（5060Ti 线），以及 llama.cpp / Neroued/ninfer / iamwavecut/ninfer-all 与社区所有贡献者。